JobShield: Code Mapping, Node Directory & Risk Scoring Weightage

Comprehensive Developer & Mentorship Guide: Exact mathematical scoring formula, severity weightage point system, and line-by-line codebase mapping with syntax-highlighted snippets.

---

1. Risk Score Calculation & Point Weightage Breakdown

JobShield computes a dynamic **0 to 100 Risk Severity Score** by evaluating detected evidence against a deterministic severity hierarchy, combined with authoritative domain registry verification signals:

Critical Fraud / Advance Fee	**Strong**	**+30 pts each**	UPFRONT_PAYMENT (deposit, registration fee, kit charge)	src/policy.py:102
Credential / Identity Harvesting	**Strong**	**+30 pts each**	SENSITIVE_INFO (OTP, bank password, Aadhaar card copy)	src/policy.py:102
Suspicious Messaging	**Moderate**	**+15 pts each**	SUSPICIOUS_MESSAGING (Telegram VIP group, WhatsApp only)	src/policy.py:104
Unrealistic Compensation	**Moderate**	**+15 pts each**	UNREALISTIC_COMP (Earn 3k-5k daily for 1 hour simple work)	src/policy.py:104
Urgency / False Deadlines	**Moderate**	**+15 pts each**	URGENCY (Pay before 5 PM, only 2 slots remaining)	src/policy.py:104
Brand Impersonation Cues	**Moderate**	**+15 pts each**	IMPERSONATION_CUES (Enterprise brand used with Gmail)	src/policy.py:104
Vague Job Description	**Moderate**	**+15 pts each**	VAGUE_DESCRIPTION (Simple tasks, no experience required)	src/policy.py:104
Personal Email Domain	**Weak**	**+5 pts each**	PERSONAL_EMAIL (@gmail, @yahoo, @hotmail, @outlook)	src/policy.py:106
Poor Writing / Grammar	**Weak**	**+5 pts each**	POOR_WRITING (Grammatical errors, unformatted claims)	src/policy.py:106
Phone-Only Contact	**Weak**	**+5 pts each**	PHONE_ONLY_CONTACT (No email or website provided)	src/policy.py:106
Company Not Found in Registry	**Penalty**	**+15 pts**	DNS host unresolved (NXDOMAIN) or RDAP status not_found	src/policy.py:111
Company Verification Ambiguous	**Penalty**	**+5 pts**	Domain active but public search returned inconclusive entity match	src/policy.py:115
Verified Company Domain Match	**Trust Bonus**	**-10 pts (Bonus)**	Active DNS host records + verified RDAP age + search match	src/policy.py:113

Exact Code Snippet: Dynamic Risk Scoring Function (src/policy.py, Lines 94–120):

```python
def calculate_risk_score(evidence: list[Evidence], company_check: CompanyCheck | None = None) -> int:
    score = 0
    for item in evidence:
        if item.rule_id.startswith('LLM_CONTEXT_'):
            score += 3    # LLM contextual signals contribute minimally
        elif item.severity == 'strong':
            score += 30   # Critical scam flags (Upfront fees, sensitive credentials)
        elif item.severity == 'moderate':
            score += 15   # Warning flags (Suspicious messaging, urgency, vague desc)
```

```
        elif item.severity == 'weak':
            score += 5    # Minor context flags (Personal email, poor grammar)

    # Company verification modifiers
    if company_check:
        if company_check.status == 'not_found':
            score += 15   # Non-existent or unresolved domain penalty
        elif company_check.status == 'verified_match':
            score = max(0, score - 10)  # Established corporate domain trust bonus
        elif company_check.status == 'ambiguous':
            score += 5    # Inconclusive domain penalty

    return max(0, min(100, score))  # Clamped strictly between 0 and 100
```

2. Decision Category Arbiter Thresholds (src/policy.py, Lines 5–92)

JobShield evaluates a deterministic decision tree in src/policy.py to categorize postings into 3 clean operational buckets:

■ HIGH RISK	**50 – 100**	• payment_request (UPFRONT_PAYMENT) • credential_request (SENSITIVE_INFO) • ≥ 2 strong flags • (strong + moderate) • (moderate + company not_found)	**DO NOT PROCEED.** Severe fraud indicators detected. High probability of financial theft or extortion.
■ NEEDS VERIFICATION	**20 – 45**	• Single moderate flag detected • Weak flags only (personal email, vague) • Company status is ambiguous or not_found without strong flags	**VERIFY DIRECTLY.** Check opening on official company portal before submitting personal documents.
■ LOW RISK	**0 – 15**	• 0 red flags detected • AND company status is verified_match with active DNS host records	**GENUINE POSTING.** Verified corporate domain. Normal applicant vigilance applies.

Exact Code Snippet: Decision Arbiter Logic (src/policy.py, Lines 22–45):

```
    credential_request = any(item.rule_id == 'SENSITIVE_INFO' for item in strong)
    payment_request = any(item.rule_id == 'UPFRONT_PAYMENT' for item in strong)
    repeated_strong = len(strong) >= 2
    repeated_moderate = len(moderate) >= 2

    if credential_request or payment_request or repeated_strong or (strong and moderate) or repeated_moderate:
        category = 'High Risk'
    elif strong and not moderate:
        category = 'High Risk'
    elif moderate and company_not_found:
        category = 'High Risk'   # Warning cues + fictitious company = High Risk
    elif moderate or weak:
        category = 'Needs Verification'
    elif not evidence and company_verified:
        category = 'Low Risk'    # 0 flags + verified corporate domain = Low Risk
```

3. Complete Component & Node Directory (Exact Code Mapping)

Below is the comprehensive guide mapping every architectural node, its source file, line numbers, and exact code snippet:

Node 0: Input Guardrails Shield & Security Clearance

File: src/guardrails.py | **Lines:** Lines 48–64
Responsibilities: Enforces character length bounds (30 to 25k characters), strips null-bytes, checks for prompt injection attacks, and rejects toxic payloads.

```
def validate_posting_text(text: str) -> str:
    clean_text = text.strip()
    if len(clean_text) < MIN_POSTING_CHARACTERS:  # min 30 chars
        raise ValueError(f'Paste a fuller job description (at least {MIN_POSTING_CHARACTERS} characters).')
    if len(clean_text) > MAX_POSTING_CHARACTERS:  # max 25,000 chars
        raise ValueError(f'The job description is too long. Limit is {MAX_POSTING_CHARACTERS:,}.')
    if '\x00' in clean_text:
        raise ValueError('Invalid null character detected.')
    is_injection, reason = check_prompt_injection(clean_text)
    return clean_text
```

Node 1: CrewAI Flow Context Engine & ReviewState

File: src/flow.py & src/pipeline.py | **Lines:** Lines 23–49
Responsibilities: Instantiates shared Pydantic ReviewState container. Carries run_id UUID, extracted entities, and evidence collections across all specialist agents.

```
class ReviewState(BaseModel):
    run_id: str = ''
    raw_text: str = ''
    posting: ParsedPosting = Field(default_factory=ParsedPosting)
    evidence: list[Evidence] = Field(default_factory=list)
    similar_postings: list[SimilarPosting] = Field(default_factory=list)
    company_check: CompanyCheck = Field(default_factory=CompanyCheck)
    workflow_events: list[str] = Field(default_factory=list)
```

Node 2: Agent 1: Posting Parser Specialist Agent

File: src/parser.py | **Lines:** Lines 18–60 & 100–145
Responsibilities: Extracts structured recruitment entities using regex & NLP: Company Name, Role Title, Contact Email/WhatsApp, Salary (LPA/CTC), and Experience (Years).

```
def parse_posting(text: str) -> ParsedPosting:
    return ParsedPosting(
        company_name=_company_name(text),        # Extracts 'Google', 'TCS', etc.
        role=_role(text),                        # Extracts 'Software Engineer', etc.
        salary=_salary(text),                    # Regex captures '12-18 LPA', 'CTC', INR
        experience=_experience(text),            # Captures '0 to 1 yrs', '3+ years'
        contact_email=_contact_email(text),      # Corporate or generic email
        contact_phone=_contact_phone(text),      # Extracted phone / WhatsApp
        links=_extract_links(text),              # URLs found in posting text
    )
```

Node 3: Agent 2: Coordinator & Routing Planner

File: src/pipeline.py | **Lines:** Lines 35–42
Responsibilities: Formulates dynamic execution plan, orders downstream specialist execution, and manages conditional branching between FastMCP tools and search fallback.

```
    # Dynamic investigation planning
    investigation_steps = ['Red-Flag Analysis']
    if posting.company_name or posting.contact_email or posting.links:
        investigation_steps.append('Company/Domain Verification')
    investigation_steps.append('Historical Vector Similarity')
    investigation_steps.append('Evidence Synthesis & Verdict')
    workflow_events.append(f'Dynamic plan formulated: {", ".join(investigation_steps)}.')
```

Node 4: Agent 3: Red-Flag Threat Scanner Agent

File: src/red_flags.py | **Lines:** Lines 109–145
Responsibilities: Executes deterministic multi-rule regex scanning across 9 threat families using finditer, capturing all co-occurring scam quotes.

```
def detect_red_flags(text: str) -> list[Evidence]:
    found: list[Evidence] = []
    for rule_id, title, severity, pattern, explanation in RULES:
        all_matches = list(pattern.finditer(text))  # Global finditer scan
        if all_matches:
            quotes = [m.group(0).strip() for m in all_matches[:3]]
            found.append(Evidence(
                rule_id=rule_id, title=title, severity=severity,
                quote=' [...] '.join(quotes),
                explanation=explanation
            ))
    return found
```

Node 5: Agent 4: Historical FAISS Vector Similarity Agent

File: src/pipeline.py & src/similarity.py | **Lines:** Lines 51–65
Responsibilities: Encodes posting into 384-dimensional vector using all-MiniLM-L6-v2. Queries FAISS vector index of 18,000+ historical labeled scams to find nearest neighbors.

```
    from .similarity import search_similar_postings
    # Embeds text via sentence-transformers/all-MiniLM-L6-v2
    raw_similar = search_similar_postings(text, top_k=4)
    if raw_similar:
        similar_postings = [SimilarPosting.model_validate(item) for item in raw_similar]
        # Computes L2 distance and cosine similarity against historical fraudulent dataset
```

Node 6: Agent 5: Company Verification Agent & FastMCP Tools

File: src/mcp_server.py | **Lines:** Lines 25–85 & 175–268
Responsibilities: Probes DNS A/AAAA host records via socket, queries ICANN RDAP registration date/age via httpx, and executes live search fallback with callback loop.

```
@mcp.tool()
def lookup_domain_dns(domain: str) -> dict:
    addr_info = socket.getaddrinfo(clean_domain, 80, proto=socket.IPPROTO_TCP)
    ips = sorted(list({entry[4][0] for entry in addr_info}))
    return {'status': 'resolved' if ips else 'no_records', 'ip_addresses': ips[:4]}

@mcp.tool()
def lookup_domain_rdap(domain: str) -> dict:
    response = httpx.get(f'https://rdap.org/domain/{query_domain}', timeout=5.0)
    return {'status': 'found', 'events': events, 'domain': clean_domain}
```

Node 7: Agent 6: Deterministic Policy Arbiter & Risk Scoring

File: src/policy.py | **Lines:** Lines 5–120
Responsibilities: Synthesizes evidence from all specialist agents. Computes final RiskDecision and dynamic 0–100 risk severity score.

```
    decision = calculate_decision(evidence, company_check)        # Evaluates decision tree
    score = calculate_risk_score(evidence, company_check)         # Computes 0-100 severity
    return AnalysisResult(
        run_id=run_id, posting=posting, evidence=evidence, decision=decision,
        company_check=company_check, duration_seconds=round(duration, 2)
    )
```

Node 8 & 9: Storage, Observability & Streamlit UI Layer

File: src/storage.py & app.py | **Lines:** Lines 870–895
Responsibilities: Persists audit trace in SQLite database and renders dynamic risk badge, 0–100 progress score bar, and external registry cards in Streamlit UI.

```
    score_val = calculate_risk_score(res.evidence, res.company_check)
    if category == 'High Risk':
        badge_html = '<div class="badge-high">■ HIGH RISK</div>'
    elif category == 'Needs Verification':
        badge_html = '<div class="badge-needs">■ NEEDS VERIFICATION</div>'
    else:
        badge_html = '<div class="badge-low">■ LOW RISK</div>'
    st.markdown(f'**Risk Severity Score:** {score_val}/100 ')
    st.progress(score_val / 100)
```